PRE

Palmas Ride Engine

How it works · How it decides · How it learns

Methodology Document

A practical guide to the data sources, scoring formula, rain detection, and accuracy-tracking that decide whether to ride Alto de Palmas tomorrow morning.

Generated June 10, 2026

1. Overview

Palmas Ride Engine is a single-purpose web app that answers one question every morning: *should I bike up Alto de Palmas tomorrow at 5:00 AM?* It blends multiple meteorological sources, weights them through a transparent scoring formula tailored to the Las Palmas climb, and produces a **0–100 score**, a **YES/NO decision**, a **confidence level**, and a list of human-readable reasons.

Beyond the live prediction, the app keeps an evolving accuracy record: every prediction is logged the night before, and the next morning the observed weather is cross-validated from two independent sources and written back so that, over time, the model's calibration is visible to the rider.

Why this exists

Open-Meteo and the SIATA weather network each have blind spots when applied to a microclimate-prone road that climbs from the Aburrá Valley (~1,500 m) to Alto de Palmas (~2,500 m). Generic forecasts often under-predict rain at altitude; SIATA stations down in the valley don't always reflect what's actually falling on the upper sections of the climb. The engine combines them deliberately so neither source alone can dictate the call.

What it is not

This is not a general weather forecast. It is narrowly tuned to the 05:00–07:30 morning ride window at Alto de Palmas, Medellín. The corridor filter, scoring weights, and time conventions are all built around that specific use case.

2. System Architecture

The app is split into four cooperating components running on Vercel.

Layer	Implementation	Role
Frontend	Static HTML / vanilla JS / Leaflet	Renders the main card, the map, the radar, and the history view.
Backend	Python on Vercel Functions (@vercel/python)	Fetches data from SIATA + Open-Meteo, computes the score, exposes /api/che
Storage	Vercel Blob (private store)	predictions.json — append-mostly log of every prediction + its observed actual.
Scheduler	Vercel Cron Jobs (twice daily)	Locks in tomorrow's prediction at 20:00 Bogota and backfills yesterday's actual

Request flow (live view)

```
Browser -> GET /api/check
           |-- fetch SIATA Pluviometrica.json    (corridor stations)
           |-- enrich: per-station {code}.json   (parallel x 15)
           |-- fetch SIATA radar animation JSON
           |-- fetch SIATA WRF forecast (Centro + Envigado)
           |-- fetch Open-Meteo forecast         (hourly precip + wind)
           |-- fetch Open-Meteo air quality      (PM2.5, PM10, AQI)
           |-- compute score, decision, reasons
           '-- best-effort save to predictions.json (Vercel Blob)
                       |
                       v
Browser renders the main card + (optionally) the More Details panel.
```

Request flow (cron)

Vercel Cron sends authenticated GET requests to `/api/cron` at fixed times. The endpoint is identical to a refresh trigger but is bearer-token gated. It runs the same data pipeline and writes the prediction or backfill regardless of whether anyone visited the site.

3. Data Sources

Five real-time inputs feed the engine. Each is queried fresh on every call (with a short in-memory cache so consecutive visits within five minutes share data).

Source	Endpoint	What we use
SIATA pluvio (summary)	`Pluviometrica.json`	Per-station current valor (rainfall snapshot).
SIATA pluvio (detail)	`{code}.json`	p10m, p1h, p24h rainfall totals + sensor freshness.
SIATA radar	`animacion_radar.json`	Last ~12 frames of precipitation echoes + bounds, animated in the More
SIATA WRF	`wrfmedCentro.json + wrfenvigado.json`	Local 24-hour rainfall forecast by quarter-day (LOW / MEDIA / ALTA / M
Open-Meteo forecast	`forecast?hourly=...`	Hourly precip, precip probability, temperature, humidity, wind speed for t
Open-Meteo air quality	`air-quality?current=...`	PM2.5, PM10, US AQI, European AQI. Informational only.

SIATA is the Sistema de Alerta Temprana de Medellín y el Valle de Aburrá — the regional weather-and-hazards network. Their public JSON endpoints are unauthenticated but use self-signed SSL, so we relax the certificate check in the fetcher.

4. The Route Corridor — which stations count

SIATA publishes hundreds of stations across the Aburrá Valley. Most are irrelevant to whether the road up Las Palmas is wet. The first job of the engine is therefore to **narrow the SIATA feed to stations whose readings actually represent Palmas weather**.

How we define "on the climb"

We pulled the live geometry of `Avenida de Las Palmas` (OSM ref=56) and `Variante Las Palmas` from OpenStreetMap via the Overpass API. The result is 186 ordered segments containing 1,883 waypoints, covering the road from its El Poblado start to past Alto de Palmas.

A SIATA station qualifies for the corridor if the minimum great-circle distance from its (lat, lon) to *any* route waypoint is **at most 1.5 km**. With waypoints this dense, that radius cleanly captures stations directly on or immediately adjacent to the road and excludes stations on the other side of the ridge.

In practice

At publication time, 15 stations pass the corridor filter. The top three by distance from the road are *Colegio Latino* (0.04 km, literally on Av. Las Palmas), *ISAGEN* (0.08 km), and *Q. La Sanin* (0.20 km). The farthest in-corridor station sits ~1.4 km off-road.

> **Why 1.5 km, not 2.5 km?** An earlier iteration used a square bounding box around the climb. That included off-route stations like Pan de Azucar (4 km north) and Las Flores in Guarne (east, past the summit) whose rain is essentially uncorrelated with Palmas. The distance-from-route filter with dense OSM waypoints lets us tighten the radius substantially without losing any genuinely-on-Palmas station.

5. The Riding Window & Timezone

The score is computed for a fixed window: **05:00–07:30 Bogota local time** on the next day. All scoring inputs are aggregated over that window only (with one exception: "overnight precipitation", which uses the hours leading *up to* 05:00).

Why Bogota time matters

Vercel functions run in UTC. Bogota is UTC–5 with no daylight saving. A naive `datetime.now()` on the server disagrees with the rider's wall clock by 5 hours, which matters at day boundaries: a cron firing at 20:00 Bogota = 01:00 UTC the next day would compute "tomorrow" as the day after the intended one. All date arithmetic therefore goes through a small Bogota timezone shim (`timeutil.py`) that anchors `today_str()` and `tomorrow_str()` to UTC–5.

6. Scoring Formula

Every prediction starts at **100**. Each input either applies a penalty (subtracts points) or a bonus (adds points). The final score is clamped to the range 0–100.

Penalties

Trigger	Threshold	Penalty
Average rain probability (05:00–07:30 from Open-Meteo)	> 60 %	**−40**
Any corridor SIATA station reports active rain	valor ≥ 0.1 mm OR p10m ≥ 0.1 mm OR p1h ≥ 0.5 mm	**−50**
Max wind in the window	> 20 km/h	**−15**
Average humidity in the window	> 90 %	**−10**
Roads classified "wet" (overnight precip > 5 mm OR active rain)	—	**−15**
Roads classified "damp" (overnight precip 1–5 mm)	—	**−7 (half)**

Bonuses

Trigger	Threshold	Bonus
Fully dry forecast window (every hour: precip < 0.1 mm AND probability < 50 %)	all hours	**+25**
No corridor station reporting rain (and at least one online)	—	**+10**

Decision & confidence

```
final_score = clamp(0, 100, sum of penalties + bonuses)

decision    = YES if final_score >= 50 else NO
confidence  = high   if score >= 70
              medium if 40 <= score < 70
              low    if score < 40
```

What is intentionally NOT in the score

- **WRF forecast** — fetched and shown in the reasons list, but contributes no points. It's information-only.
- **Radar imagery** — fetched, displayed in More Details, but never weighted. It's a visual aid.

- **Air quality (PM2.5, PM10, AQI)** — fetched and shown with a non-zero color badge, but explicitly disclaimed as "not factored into the score."

> The score is a *heuristic*, not a calibrated probability. A score of 100 does not mean a 100% chance of dry roads. It means: no penalty triggered, the dry-window bonus applied, no station reported rain. The empirical mapping from score to actual dry-ride probability is what the accuracy log is gradually filling in.

7. Rain Detection — Four Signals

A naive "is it raining now" check on SIATA's pluviometric data was responsible for two real-world misses: a sub-millimeter zombie reading from one station permanently anchored the score at 50, and a real rainy Saturday morning slipped past entirely because the summary `valor` field reported zero between brief showers.

The fix was to stop treating `valor` as the only signal. Every corridor station now exposes **four** independent rainfall signals; a station is flagged "raining" if **any** of them crosses its threshold:

Signal	Window	Source	Raining threshold
`valor`	current snapshot	`Pluviometrica.json`	≥ 0.1 mm
`p10m`	last 10 min	`{code}.json`	≥ 0.1 mm
`p1h`	last 1 hour	`{code}.json`	≥ 0.5 mm
`p24h`	last 24 hours	`{code}.json`	informational only (used for road-wetness)

Sub-mm noise threshold

Values below 0.1 mm are treated as no rain. SIATA's pluvio stations regularly report tiny non-zero residuals (e.g. 0.02 mm) on dead sensors that haven't been re-zeroed. Below this threshold we ignore the reading rather than let one zombie sensor anchor the entire score.

Sensor-liveness cross-check

Even with the 0.1 mm threshold, SIATA could in principle serve a stale `0.5 mm` residual from a sensor whose rainfall hardware is actually offline. The cross-check fetches the station's per-station detail file: if all three of `p10m`, `p1h`, and `p24h` are `-999`, the rain sensor is treated as offline and the value is dropped.

Detail files are fetched in parallel for all 15 corridor stations using a `ThreadPoolExecutor` with a 3-second per-station timeout. Total enrichment latency is typically ~0.7 s. Each file is cached for 5 minutes.

8. Road Condition Inference

Road wetness is computed separately from "is it raining" because the surface state at 5:00 AM depends on what fell overnight, not on what's falling at the moment of the prediction (8:00 PM the night before). Two independent sources are combined:

Source	Window	Used as
Open-Meteo overnight precip	20:00 today → 04:59 tomorrow (Bogota)	Sum of hourly precip
SIATA p24h average	rolling last 24 h across corridor	Mean of corridor stations' p24h

The worst case wins. The two sources are combined by taking `max(open_meteo_overnight, siata_p24h_avg)` and bucketing the result:

```
max(om_overnight, siata_p24h_avg)  →  road condition
  > 5 mm   →  wet
  1–5 mm   →  damp
  0.2–1 mm →  mostly_dry
  ≤ 0.2 mm →  dry
```

If SIATA captures rain that Open-Meteo's interpolated grid missed — the classic Palmas microclimate situation — SIATA wins and the road is flagged damp or wet accordingly. The reverse also works: if the Open-Meteo grid sees rain that SIATA stations down in the valley missed, Open-Meteo wins.

9. Recording the Actual — Dual-Source Backfill

Each prediction is logged at 20:00 Bogota the night before. The morning after the ride window closes, the engine records what *actually* happened, using the same two sources blended the same way:

```
open_meteo_precip = sum(forecast.past_days['precipitation']
                        for hours 05:00–07:00 of the ride day)
siata_p24h_avg    = mean(corridor stations' p24h at 07:00 cron time)
                    # p24h covers ~yesterday afternoon → 07:00 = the
                    # ride window plus overnight

actual.precip_mm  = max(open_meteo_precip, siata_p24h_avg)
actual.rained     = actual.precip_mm > 0.1
```

Source label & disagreement

Every backfilled actual carries an explicit `source` field so a reader can audit which input dominated the call:

- **`agreed`** — both sources within 0.5 mm of each other.
- **`open_meteo`** — Open-Meteo recorded more rain than SIATA.
- **`siata_p24h`** — SIATA recorded more rain than Open-Meteo's grid. **These are the interesting ones**: cases where the microclimate showed up at the on-road stations but missed the broader forecast model.

Why "max" instead of average?

When the two sources disagree by a meaningful amount, one of them is almost always closer to ground truth. The expected error of averaging is symmetric (under-reporting and over-reporting are equally bad), but for a cyclist's decision, *under-reporting* is the worse failure mode (it tells you to ride when you shouldn't). Taking the max biases toward over-reporting, which is the safer asymmetry to carry.

10. The Audit Trail

Every prediction in `predictions.json` stores a complete snapshot of the sensor state at the time of the prediction *and* at the time of the backfill. This lets a reader inspect any past entry and see exactly what each station was reporting, and what each external source said about the morning in question.

Schema (one entry)

```
{
  "ride_date": "2026-06-11",
  "predicted_at": "2026-06-10T01:00:00Z",
  "score": 92, "decision": "YES", "confidence": "high",
  "reasons": ["Moderate/low rain probability (8%)", "Low wind ..."],
  "forecast": {
    "avg_precip_prob": 8.0, "max_wind": 4.2,
    "avg_humidity": 96.0, "overnight_precip_mm": 0.3,
    "station_snapshots": [   // captured at predict time
      {"name": "Colegio Latino", "code": 251,
       "distance_km": 0.04,
       "valor": 0.0, "p10m": 0.0, "p1h": 0.0, "p24h": 0.6},
      ...   // one per corridor station
    ]
  },
  "actual": {            // filled in the next morning by cron
    "precip_mm": 5.1, "rained": true, "max_wind": 6.3,
    "open_meteo_precip_mm": 0.0,
    "siata_p24h_avg_mm": 5.1, "siata_p24h_max_mm": 7.3,
    "disagreement_mm": 5.1,
    "source": "siata_p24h",
    "station_snapshots": [   // captured at backfill time
      {"name": "Colegio Latino", "code": 251,
       "valor": 0.0, "p10m": 0.0, "p1h": 0.0, "p24h": 7.3},
      ...
    ]
  },
  "correct": false   // YES + rained = wrong call
}
```

Source labels surface directly in the History view as small badges (Open-Meteo / SIATA / Agreed) so a SIATA badge on a wrong prediction is the visual signal that microclimate rain was the specific failure mode.

11. Automation

Two Vercel Cron Jobs guarantee the daily prediction and backfill happen whether anyone visits the site or not. Both hit the same endpoint (`/api/cron`), which always runs both jobs — they're idempotent.

Bogota	UTC cron	What it does
20:00	`0 1 * * *`	Locks in tomorrow's prediction. The prediction is computed and written to predictions.json.
07:00	`0 12 * * *`	Backfills the just-finished ride's actual using SIATA + Open-Meteo (dual-source).

Cron security

`/api/cron` is gated by `CRON_SECRET`, a strong random value stored as a Vercel environment variable. Vercel attaches `Authorization: Bearer ${CRON_SECRET}` to its outbound cron requests; the endpoint verifies and returns 401 otherwise. The user-facing endpoints (`/api/check`, `/api/history`) stay unauthenticated so the browser UI works without secrets.

Why redundant logging?

`/api/check` and `/api/history` also opportunistically save predictions and backfill actuals — mirroring exactly what cron does. That means the system stays current even if cron has a hiccup, as long as someone visits the site. Writes are upserts keyed by ride_date and refuse to overwrite once an actual is recorded, so the redundancy is safe.

12. UI Tour

Main view

The primary card shows tomorrow's **vibe line** (a playful one-liner keyed to the score), the numeric **score** with a gradient bar, **confidence**, the **05:00–07:30 riding window**, the inferred **road conditions**, the human-readable **reasons** that fed the score, and the live status of each **data source**.

More Details

A toggle expands the detail panel, which is purely informational:

- **Air Quality** — current PM2.5, PM10, US AQI, European AQI from Open-Meteo. Color-coded badge using the standard EPA scale.
- **Radar** — the last ~12 SIATA radar frames overlaid on an OpenStreetMap of the Aburrá Valley, cycling every 600 ms. The route polyline is drawn on top so you can read precipitation against the actual road.
- **Stations on the Climb** — Leaflet mini-map showing each corridor station as a colored pin (green = dry, blue = active rain, gray = offline), with the route polyline. Tapping a pin pops up the station's name, neighborhood, distance from the route, and all four rain signals (now, 10m, 1h, 24h).

History

Stats card (accuracy %, correct count, wrong count, pending count) plus a list of recent predictions with verdict (correct / wrong / pending), source badge (Open-Meteo / SIATA / Agreed), and the score in the route's accent color.

Language toggle

A circular EN/ES button fixed to the top-right corner toggles the UI between English and Colombian Spanish. The choice persists in `localStorage`. Dates use the matching locale (en-US vs es-CO). Backend-generated reasons currently stay in English regardless — see Limitations.

13. Refresh & Caching Semantics

Three caching layers exist; understanding them prevents the surprise of "I clicked refresh but nothing changed."

Layer	TTL	Bypassed by
Vercel CDN (in front of static assets)	Standard immutable for /static/*; /api/* is already cache-busted	N/A (already cache-busted)
In-memory cache (cache.py, per Vercel function instance)	5 minutes (300 s)	Refresh button (?fresh=1) clears it explicitly.
Vercel Blob CDN (in front of predictions.json)	~60 seconds even with cache-control: no-cache	Etag-based cache-busting query string: ?v=<etag>

The **Refresh button** is the user-facing escape hatch: clicking it sends a request with `?fresh=1` and `cache: "no-store"`. The server clears its in-memory cache before fetching, and the blob read uses the etag cache-buster so the just-written prediction is visible immediately. The **Last updated** timestamp on the main card makes any successful refresh visible even when the underlying numbers haven't changed.

14. Limitations & Known Trade-offs

- **SIATA detail files are rolling.** Their per-station endpoints expose only p10m / p1h / p24h — there's no archive. So we cannot retroactively reconstruct a past day's rain from SIATA. Going forward, the 07:00 cron captures p24h daily; before that, history is limited to what Open-Meteo's grid recorded.
- **Backend-generated reasons are English-only.** The score formula returns fully-formed sentences ("Moderate/low rain probability (28%)"). Localizing them would require refactoring analyze.py to return structured keys and formatting on the frontend.
- **The score is heuristic, not probabilistic.** A score of 100 is not P(dry) = 1.0. With enough logged predictions + actuals, calibration becomes possible — that's what the audit trail is preparing for.
- **Vercel Hobby tier function timeout is 10 s.** Current /api/check runtime is ~5 s in the worst case (15 parallel SIATA detail fetches + Open-Meteo + radar JSON). There's headroom but not infinite.
- **WRF and radar are not in the score.** They are shown to the rider but never weighted. A future refinement could parse WRF's morning rain levels (BAJA / MEDIA / ALTA / MUY ALTA) into a small penalty.

These are deliberate boundaries, not unfixed bugs. Each one is a tradeoff: more sources = more latency and more places for one signal to dominate; full localization = more code surface; calibrated probabilities require a much larger labelled dataset than we currently have.

15. Future Work

- **Calibrated probability display.** Once the log holds ~30 days of evaluated predictions, fit a simple isotonic-regression calibrator from score → P(dry) and display the calibrated number alongside the raw score.
- **Score auto-tuning.** The decision threshold is hardcoded at 50. With a labelled history, that threshold can be set to whatever maximizes correct calls on the rider's own data.
- **Per-source accuracy breakdown.** Which signal predicts best — Open-Meteo's precip probability, SIATA's overnight readings, the WRF forecast? Splitting accuracy by source would reveal which inputs are carrying the weight.
- **WRF in the score.** Add a small penalty for MUY ALTA / ALTA in the morning forecast, since it currently contributes nothing.
- **Push notification at 20:30 Bogota.** When the cron finishes the 20:00 prediction, optionally fire a push or email summarizing tomorrow's call so the rider doesn't have to open the site.
- **Backend reason localization.** Return structured keys with params instead of pre-formatted sentences, so the EN/ES toggle covers the reasons too.

Appendix A — Configurable Knobs

All of these live in `config.py` and can be adjusted without touching the rest of the codebase.

Knob	Default	What it controls
`RIDE_EARLIEST`	**5**	First hour of the ride window (Bogota)
`RIDE_LATEST`	**7**	Hour the ride window ends (07:00–07:59 inclusive)
`CORRIDOR_RADIUS_KM`	**1.5**	Max km a station can sit off-route to count
`SCORING.rain_prob_high_threshold`	**60 %**	Threshold above which the rain-probability penalty fires
`SCORING.rain_prob_high_penalty`	**–40**	Penalty applied above the threshold
`SCORING.active_rainfall_penalty`	**–50**	Applied when any corridor station reports rain
`SCORING.high_wind_threshold`	**20 km/h**	Above this max wind triggers the wind penalty
`SCORING.high_wind_penalty`	**–15**	Applied above the wind threshold
`SCORING.high_humidity_threshold`	**90 %**	Above this avg humidity triggers the humidity penalty
`SCORING.high_humidity_penalty`	**–10**	Applied above the humidity threshold
`SCORING.dry_window_bonus`	**+25**	All hours in window must be precip < 0.1 mm AND prob < 50 %
`SCORING.no_recent_rain_bonus`	**+10**	No corridor station reporting rain
`SCORING.wet_road_penalty`	**–15**	Applied when roads are inferred wet (half for damp)
`CACHE_TTL_SECONDS`	**300**	How long the in-memory cache holds each SIATA / OM response

Appendix B — HTTP Endpoints

Endpoint	Method	Auth	Purpose
`/api/check`	GET	`none`	Compute & return tomorrow's prediction. Side effect: opportunistic save + backfill. S
`/api/history`	GET	`none`	Return the prediction log + accuracy stats. Side effect: opportunistic backfill of pend
`/api/cron`	GET	`Bearer CRON_SECRET`	Cron-only endpoint. Always logs tomorrow's prediction AND backfills pending actua
`/api/health`	GET	`none`	Simple liveness probe.

Repository

github.com/bensykora997/palmasridechecker

File map

```
config.py             # Tuning knobs (corridor radius, scoring weights)
palmas_route_data.py  # Avenida de Las Palmas geometry from OSM
timeutil.py           # Bogota timezone shim
fetch_siata.py        # SIATA pluvio + radar + WRF
fetch_openmeteo.py    # Open-Meteo forecast + archive
fetch_air.py          # Open-Meteo air quality
analyze.py            # Score + road conditions + station analysis
cache.py              # 5-minute in-memory cache
db.py                 # Vercel Blob client (predictions.json)
api/check.py          # GET /api/check
api/history.py        # GET /api/history
api/cron.py           # GET /api/cron (Bearer auth)
api/health.py         # GET /api/health
static/               # index.html + app.js + style.css (vanilla + Leaflet)
vercel.json           # Routes + cron schedule
```

Buenas pintas y a clavar, parcero.